

Urban Mobility Optimization System (UMOS): A Systems Analysis and Design Approach for University Transport in Bogotá

Natalie Mariño Figueroa – 20232020143, nmarinof@udistrital.edu.co¹, Edilson Santiago Sepúlveda Cortés – 20231020237, essepulvedac@udistrital.edu.co¹, Marlon Yecid Riveros Guio – 20231020208, myriverosg@udistrital.edu.co¹, and Juan Esteban Cañon Solorza – 20232020078, jecanons@udistrital.edu.co¹

¹Dept. of Computer Engineering, Universidad Distrital Francisco José de Caldas

ABSTRACT

This paper covers the full cycle of analysis, design, and planned implementation of UMOS, a multimodal route planning platform for the community of the Universidad Distrital Francisco José de Caldas in Bogotá. We looked at the urban transport network around two campuses through the lens of systems theory (entropy, synergy, sensitivity, deterministic versus stochastic behavior) and found that, on paper, it behaves like a textbook complex adaptive system. The analysis was grounded in 312 survey responses, 20 hours of stop-level observation, and 5 full days of gate monitoring at both campuses during February 2026. From that data, we moved into design, producing a microservices architecture organized into seven modules and formalized with UML diagrams. For the route engine, we chose Ant Colony Optimization over classical shortest-path algorithms because Bogotá's traffic is stochastic enough that fixed-weight approaches fall apart within minutes. This paper brings together two workshop deliverables and outlines a 20-week path to a working prototype before Semester 2026-I ends.

Keywords: Systems analysis, urban mobility, ant colony optimization, complex systems, entropy, requirements engineering, TransMilenio, Bogotá

I. INTRODUCTION

Bogotá is not a kind city to commuters. Moovit's 2024 transit index puts the average public transport trip at 64 minutes [3], and for students at the Universidad Distrital Francisco José de Caldas (UDFJC), it is worse than that. We surveyed 312 people between students, faculty, and administrative staff, and the average one-way commute came out to 67 minutes. Students who combine TransMilenio with SITP feeders reported 82. Then there is rain. On wet mornings, wait times climb by 63%, cyclists essentially disappear (down 93.5%), and SITP buses run at 108% occupancy. Those are not edge cases; they are typical rainy-day conditions in a city that sits at 2,600 meters and gets rain about a third of school days.

That is the problem UMOS tries to address. The idea is a mo-

bile app that gives the UDFJC community real-time multimodal route recommendations, pulling from TransMilenio BRT, SITP urban buses, TransMiCable, cycling infrastructure, and pedestrian paths. Nothing revolutionary on its face. But when we sat down to actually design it, we realized the transport system we needed to optimize had properties that made straightforward engineering insufficient.

This paper brings together two phases of work. The first workshop was about analysis: tearing apart the transport network into subsystems, cataloging its elements, going out and collecting data, finding feedback loops. The second workshop moved into design: writing formal requirements, drawing UML diagrams, picking technologies, planning a roadmap. We also chose the Ant Colony Optimization algorithm for route calculation, which is probably the most unusual technical decision in the project and one we spend a full section justifying.

II. THEORETICAL FRAMEWORK

We framed the whole project around systems theory. Not because the syllabus told us to (though it did help), but because the transport network really does behave like the systems that Meadows [7] and Strogatz [8] write about. Below we walk through the specific concepts that ended up mattering most for our design choices.

A). *Open System, Closed Modules*

An open system exchanges both matter and energy with its environment. The UMOS fits this description almost too well. It receives inputs from across the city: traffic conditions on Carrera Séptima, Pico y Placa restrictions, weather fronts rolling in from the sabana, construction closures from IDU, fare policies from TransMilenio S.A., and demand spillover from the four or five other universities whose students use the same corridors. It also produces outputs that affect people well beyond the UDFJC campus, since 22,900 undergraduates flooding TransMilenio stations at 07:30 move the needle on service levels for everyone.

Now here is the part that surprised us in the design phase.

Even though the UMOs as a whole is open, we found it useful to make some software modules behave like closed systems. The User Management module, for example, handles authentication and profiles without touching any real-time external feed. This gives it stable, predictable behavior regardless of what is happening with weather APIs or GPS streams. The Route Optimization Engine, by contrast, is as open as it gets, constantly absorbing live data from every source we can pipe into it. Balancing those two approaches, closed where stability matters and open where adaptability matters, turned out to be one of the more interesting design tensions we had to resolve.

B). Entropy

Shannon [1] gave us a mathematical framework for measuring disorder and unpredictability. We borrowed the concept loosely and applied it to the student commute experience. On a typical dry morning, entropy is low: buses more or less follow their schedules, travel times cluster around predictable values, the system behaves almost deterministically. On a rainy Wednesday with a march on the Séptima and some university event at Sabio Caldas, entropy shoots up. Buses start bunching, wait times scatter randomly, and anyone relying on a fixed schedule is in trouble.

We measured this shift. The coefficient of variation for arrival times at the main SITP stop went from 0.18 on dry days to 0.41 when it rained, a 128% jump in relative variability. The system, in other words, becomes about twice as hard to predict.

UMOS cannot change the weather or fix the roads. What it can do is give people better information, and information is the classical antidote to entropy. If a student knows the next bus is 14 minutes away but an alternative route arrives in 6, the trip becomes less uncertain even though the system itself is still messy. One specific consequence of this thinking: the Workshop #2 requirement that every piece of data in the app must display its age (“Updated 45s ago”). We added this because 39% of surveyed students said they distrusted the timeliness of existing mobility apps.

C). Why Dijkstra Was Not Enough

A deterministic system gives the same output for the same inputs every time. Urban transit does not work that way. Two identical Tuesday mornings will produce different passenger counts at Estación Universidades at 07:45 because someone overslept, someone decided to bike instead, someone got off at the wrong stop. The system has tendencies, sure. The peak always comes. Demand always drops after 09:00. But the exact numbers bounce around.

We will get into ACO in more detail in Section VII. The short version is that Dijkstra’s algorithm treats edge weights as fixed numbers, which works fine when you have a static graph. On the Ak 7 corridor, travel times swing from 8 to 22 minutes depending on the time of day, whether it is raining, and whether someone rear-ended a bus on Calle 45. An algorithm that assumes fixed weights is computing yesterday’s optimal route.

D). Sensitivity

The sensitivity analysis from Workshop #1 (Table 3) turned up something we had not expected: the system’s behavior is dominated by just two or three parameters. A 10% uptick in rainfall intensity leads to a 26.4% increase in wait times. That is non-linear amplification, not a proportional response. SITP bus frequency has a similar outsized effect (11.2% punctuality shift per 10% frequency change). What makes it worse is that these two interact. Rain slows buses, which lowers effective frequency, which compounds the wait increase. You get a positive feedback loop from a single weather event, and the system takes 45 to 60 minutes to recover after the rain stops.

E). Synergy

Here is something we did not plan to find. Students had already built their own optimization system, of sorts. WhatsApp groups where people share bus arrival times, warn about delays, post photos of crowded stations. No individual student had enough data to plan an optimal route, but the group collectively produced mobility intelligence that outperformed any single official source. That is synergy: the whole exceeding the sum of its parts, except it was emergent and informal.

The design goal for UMOs is to formalize that process. Instead of a hundred WhatsApp groups with patchy coverage, one platform that converges GPS data, weather feeds, transit schedules, and user-submitted reports into something more reliable. The architecture reflects this: every module feeds the Route Engine, which mixes all the inputs together before making a recommendation. You cannot get a good route by looking at bus schedules alone or traffic alone or weather alone. You have to look at all of it at once. That is what people mean when they say holistic design, although we hesitate to use the term because it sounds like a wellness blog.

III. DATA COLLECTION

We collected data during weeks 1 through 3 of February 2026 at both the Sabio Caldas and Tecnológica campuses. Three methods: direct observation at the main SITP stops (Ak 7/Cl 40b and Ak 13/Cl 40) plus campus entrances, 20 hours total over 8 weekdays in 15-minute intervals; a 12-question Google Forms survey distributed via QR codes at campus entries, yielding 312 responses (78.5% undergrads); and manual gate counting every 10 minutes across 5 full academic days. Tables 1 and 2 summarize the modal split and the rain effect.

Table 1: Transport Mode Distribution (N=312)

Mode	N	%	Avg. Time
SITP Urban Bus	118	37.8	58 min
TransMilenio BRT	79	25.3	71 min
TM + SITP combined	54	17.3	82 min
Private vehicle	28	9.0	47 min
Bicycle	19	6.1	31 min
Walking only	14	4.5	18 min

Table 2: Rainy vs. Dry Day (07:00–09:00)

Metric	Dry	Rainy	Delta
Total arrivals	874	621	−28.9%
Avg. wait time	19 min	31 min	+63.2%
Bicycle arrivals	62	4	−93.5%
SITP occupancy	74%	108%	+45.9%
Late reports	11.4%	38.7%	+27.3pp

IV. ANALYSIS RESULTS

A). Structure and Performance

We broke the UMOs into four subsystems (Physical Transport, Fleet and Operations, Information and Digital, Governance) and cataloged 24 critical elements across them. The overall performance score came out to 12 out of 25, which is bleak. Efficiency got 2/5. Reliability, 2/5. Equity, also 2/5. The equity number is worth pausing on: because the fare is a flat COP \$3,550 regardless of distance, students commuting from Ciudad Bolívar in the south end up paying 3.4 times more per kilometer than students from Chapinero who live a few blocks from campus.

Table 3: Parameter Sensitivity (10% Change Effect)

Parameter	Wait	Punctuality	Class
Rainfall intensity	+26.4%	−18.1%	Critical
SITP bus frequency	−14.8%	+11.2%	High
Passenger demand	+18.3%	−13.6%	High
Ak 7 traffic density	+11.7%	−8.8%	Medium
SITP App adoption	−4.9%	+3.7%	Medium
Schedule spread	−9.2%	+7.4%	Medium

B). Feedback Loops and Emergent Behavior

Four feedback loops stood out. The first is bus bunching, a well-documented transit phenomenon [9] that we observed firsthand on Ak 7: a delayed bus accumulates extra passengers at each stop, which slows it further, which means the bus behind it finds empty stops and catches up, and eventually two or three buses arrive together followed by a long gap. The second loop is the rain cascade. When it rains, bicycle commuters switch to buses, but the buses are already full, so boarding takes longer, which triggers the bunching from loop one. These two loops feed each other.

On the balancing side, students who use real-time apps adjust their departure times, which spreads demand out a little. Bus drivers also compensate informally by adjusting speed and dwell time, though this is inconsistent and depends entirely on individual behavior.

We also found some genuinely emergent stuff. The WhatsApp groups, obviously. But also spontaneous queue formation at Estación Universidades during peak hours: students line up in orderly fashion without any signage or instructions telling them to, but only when the station is crowded. During off-peak hours, there is no queue behavior at all. And then there was the

Wednesday anomaly, a 34% spike in arrivals at 10:15 that we eventually traced to multi-section courses all starting at 10:00 on Wednesdays.

C). Bottlenecks

Table 4: System Bottlenecks

#	Bottleneck	Type	Impact
B1	Ak 7 mixed traffic (CI 36–45)	Struct.	8–14 min, affects 52%
B2	Est. Universidades single hub	Struct.	Single point for 29%
B3	08:00 class synchronization	Regul.	Demand > capacity
B4	Campus gate processing	Oper.	4–7 min queues
B5	No weather-resilient last-mile	Struct.	Overflow to buses
B6	TransMiCable single access	Struct.	Isolates ~22%

V. REQUIREMENTS

In Workshop #2, we took the messy analytical findings and forced them into the structured language of software requirements. Six categories of constraints frame the design space: the physical limitations of road infrastructure, the demand peaks that exceed transport capacity, structural vulnerabilities like single transfer nodes, weather sensitivity, regulatory constraints like the flat fare and Pico y Placa, and gaps in real-time data availability.

On the functional side, the system needs to let users input origin and destination, compute routes from live traffic data, integrate across modes (BRT, bus, bicycle, walking), push delay and disruption alerts, suggest alternatives during congestion, estimate travel times per option, and support multimodal trip combinations.

Non-functional requirements include usability (simple mobile UI), performance (route results in under 2 seconds), reliability (consistently fresh data), 99.5% availability, device accessibility, and security compliant with Ley 1581 [13].

Two requirements came out of the analysis that do not appear in standard software engineering textbooks. One is equity: the system should actively work to reduce the mobility gap between students from different neighborhoods, especially the 3.4× cost-per-km disparity facing southern commuters. The other is forward scalability to accommodate the Metro de Bogotá, whenever that eventually launches.

VI. ARCHITECTURE

A). Module Structure

The architecture splits into seven modules: Trip Capture (receives origin, destination, time, preferences), External Data Integration (consolidates feeds from TransMilenio, SDM traffic, IDEAM weather, IDU construction, UDFJC academic system), Route Optimization Engine (the ACO-based core), Notifica-

tion Management (push alerts via Firebase), Presentation (React Native mobile app plus web client), Persistence and Analytics (PostgreSQL/PostGIS, Redis, TimescaleDB), and Policy/Rules (fare constraints, Pico y Placa logic, schedule-based demand shaping). Each module maps to one or more NestJS microservices talking through REST, gRPC, and Redis event streams.

B). UML Formalization

We produced five diagrams in Workshop #2. The context diagram places UMOS at the center with five external systems and three user types. The logical architecture shows how modules hand off data internally. The component diagram names the concrete software pieces: TripRequestController, RoutingEngine, TrafficDataAdapter, WeatherDataAdapter, NotificationService, MobilityDB, PolicyEngine. Each has explicit interfaces like `computeOptimalRoute()` or `sendAlert()`. The sequence diagram traces a full “Plan Trip” request from user tap to backend response. And the deployment diagram maps everything onto GKE nodes behind a Kong API Gateway, with managed database services and secure external API connections.

C). Tech Stack

Node.js 20 with TypeScript and NestJS on the backend (event-driven, handles the roughly 14,000 GPS events per minute at peak). Socket.IO for real-time push. React Native with Expo for the mobile app, Mapbox GL for maps. PostgreSQL/PostGIS for geospatial storage, Redis for caching with TTLs from 30 seconds to 5 minutes, TimescaleDB for historical time-series. All of it running on GCP’s Kubernetes Engine, deployed via GitHub Actions and ArgoCD.

VII. ROUTE OPTIMIZATION WITH ACO

Classical shortest-path algorithms assume you know the edge weights and that they stay put. In Bogotá, neither is true. We went with Ant Colony Optimization [10] because it was designed for exactly the kind of problem we have: stochastic weights, multiple objectives, and a graph that changes while you are traversing it.

The basic mechanism is this: artificial ants walk through a graph representing the multimodal network (~ 180 nodes, ~ 420 edges). Each ant chooses the next node probabilistically, influenced by pheromone τ_{ij} (which encodes past success) and a heuristic η_{ij} (inverse of current travel time):

$$P_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{k \in \mathcal{N}_i} [\tau_{ik}]^\alpha \cdot [\eta_{ik}]^\beta} \quad (1)$$

After a generation of ants finishes, pheromone evaporates and gets replenished based on route quality:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k \quad (2)$$

The evaporation rate ρ is the piece that matters most for our use case. We want the algorithm to forget. If a route was

great yesterday but Ak 7 has an accident today, clinging to stale pheromone trails is actively harmful. We set $\rho = 0.15$, which means about 15% of historical information decays each cycle. During rainy mornings, we crank up β to give more weight to current conditions; during stable periods, α goes up to rely more on accumulated experience. Fifty ants per query, 20 iterations, convergence under 800 milliseconds on a single GKE node.

VIII. RISK MANAGEMENT

Five things can go wrong, and we planned for each. External APIs (TransMilenio, SDM, weather, IDU, academic system) can and will go down. Kong Gateway runs circuit breakers: three failures in a row and the system falls back to Redis-cached data for a minute before retrying. The Universidades station bottleneck showed us what single points of failure look like in the physical system; in software, we keep microservices isolated so a notification crash does not take down route calculation. Weather feeds into the routing engine as a first-class variable, not an afterthought, because our data showed rainfall is the single most disruptive parameter. Every piece of information in the app carries an age tag because users told us they do not trust data whose freshness they cannot verify. And if GTFS-RT feeds die completely, the system falls back to static schedules rather than going blank.

IX. QUALITY AND PERFORMANCE

The morning peak hits about 128 arrivals every 15 minutes. Kubernetes auto-scales the route engine up during that window and back down after 09:30 to keep the monthly bill around \$780 (covered by GCP academic credits). We target 99.5% uptime through blue-green deployments: deploy to the standby environment, run automated checks, flip traffic at Kong, roll back instantly if something breaks. Deployments only happen during the quiet windows we identified in the data (mid-morning or after 20:00).

On the code side: ESLint with Airbnb rules, TypeScript in strict mode, 80% test coverage for routing logic and 60% for supporting modules. Security is OAuth 2.0 with short JWTs, TLS 1.3 everywhere, AES-256 for stored data, and a 48-hour auto-delete policy on raw location traces to comply with Colombia’s data protection law [13].

X. IMPLEMENTATION ROADMAP

Twenty weeks, ten sprints, four phases. Phase 1 (weeks 1–4) lays the infrastructure and builds user management with SSO. Phase 2 (weeks 5–8) connects external data sources and builds the traffic monitoring module, including the rain classifier. We front-loaded weather and traffic because the sensitivity analysis told us those are the parameters where improvement matters most. Phase 3 (weeks 9–14) is the core: ACO route engine, travel time estimation, multimodal integration, notifications, and the mobile app MVP going into beta at week 14. Phase 4 (weeks 15–20) adds prediction models, personalization, community alerts, accessibility (WCAG 2.1 AA), security hard-

ening, and production launch.

XI. FINAL PRODUCT

What we deliver at the end of the semester is a mobile app, available for Android and iOS, that UDFJC students and staff can download and use. It will show bus positions on an interactive map, calculate multimodal routes using the ACO engine with live data, send push notifications about delays and rain-triggered mode shifts, display estimated travel times with confidence ranges rather than single numbers, let users save favorite routes and optimize departure based on their class schedule, and include a layer where people can report crowded buses, blocked roads, or long campus gate queues.

We also deliver this paper, a poster, and a set of presentation slides. The codebase and architecture documentation go on a public GitHub repository so future students can pick up where we left off.

We are not going to fix Bogotá's traffic with a mobile app. That requires billions of pesos in infrastructure and political decisions nobody in our team has any influence over. What we might be able to do is shave a few minutes off someone's commute, or at least reduce the anxiety of not knowing when the next bus is coming. If a student who currently averages 67 minutes gets that down to 58, or even just stops missing the 08:00 lecture because the app told them to leave five minutes earlier, that is worth the effort.

XII. CONCLUSIONS

We started this project thinking we were building a routing app. What we actually ended up doing was a systems analysis exercise that happened to produce an app design. The transport network around UDFJC is complex, not just complicated. It has feedback loops that amplify small disruptions into system-wide delays. It has emergent behaviors like spontaneous queuing that nobody designed or planned. Its most important parameters interact non-linearly. And it is stochastic enough that any tool treating it as a deterministic problem will fail.

Those properties drove the big design choices. Microservices because different parts of the system need different things at different times. ACO because deterministic routing breaks in a stochastic environment. Push-based updates because the system's state changes too fast for polling. Phased delivery because you learn more from real users than from requirements documents, no matter how carefully you write them.

The prototype at week 20 will be the first real test. Maybe the TransMilenio API breaks every other Tuesday. Maybe nobody downloads the app because WhatsApp is good enough. Maybe the ACO parameters we picked on synthetic data do not hold up in the real network. We do not know yet. What we do know is that we understood the problem deeply enough to make engineering decisions that respect its complexity rather than pretending it away, and that is something.

REFERENCES

- [1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [2] TransMilenio S.A., "Estadísticas de oferta y demanda del SITP, dic. 2024," Bogotá, 2024.
- [3] Moovit Insights, "Public Transit Index – Bogotá," 2024.
- [4] Secretaría Distrital de Movilidad, "Informe de movilidad en Bogotá 2023," 2024.
- [5] G. Booch, J. Rumbaugh, I. Jacobson, *The UML User Guide*, 2nd ed. Addison-Wesley, 2005.
- [6] K. Wiegers, J. Beatty, *Software Requirements*, 3rd ed. Microsoft Press, 2013.
- [7] D. H. Meadows, *Thinking in Systems: A Primer*. Chelsea Green, 2008.
- [8] S. H. Strogatz, *Nonlinear Dynamics and Chaos*, 2nd ed. Westview, 2015.
- [9] V. R. Vuchic, *Urban Transit Systems and Technology*. Wiley, 2007.
- [10] M. Dorigo, T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
- [11] S. Newman, *Building Microservices*, 2nd ed. O'Reilly, 2021.
- [12] Alcaldía Mayor de Bogotá, "Decreto 309 de 2009," Sec. Jurídica Distrital, 2009.
- [13] Congreso de Colombia, "Ley 1581 de 2012," 2012.
- [14] GTFS.org, "General Transit Feed Specification Reference," 2024.